
ProctorShield AI Final Defense Study Guide

System Audit Architecture Security Testing and Defense Preparation

ProctorShield AI

AI Assisted Online Examination and Proctoring System

Prepared for the ProctorShield AI Project Team

Audited 9 September 2026

Baseline main commit a4d81bc plus current local mobile and real time warning changes

How to Use This Guide

This guide is both a codebase audit and a defense reviewer. Study the executive summary first, rehearse the demonstration plan, then practice answering the panel questions without reading. The wording deliberately separates implemented controls from claims that still require research evidence.

Important defense rule. Do not claim that browser proctoring makes cheating impossible. Present ProctorShield AI as a layered risk-detection, evidence-capture, and human-review system with documented device limitations.

Contents

1 Executive Defense Summary	Core story
2 Problem Scope and Users	Boundaries
3 System Architecture	Components
4 Core Workflows	Behavior
5 Proctoring and Evidence Design	Detection
6 Real Time and Mobile Design	Capabilities
7 Data Model	Persistence
8 Security Model	Controls
9 Subscription and Billing	Entitlements
10 Verification Evidence	Tests
11 Prioritized Audit Findings	Fix order
12 Claims to Avoid During Defense	Accuracy
13 Demonstration Plan	Live demo
14 Defense Presentation Script	Talk track
15 Likely Panel Questions	Practice
16 Additional Technical Questions	Deep dive
17 Final Defense Checklist	Readiness
18 Command Reference	Operations
19 Glossary	Terms

1 Executive Defense Summary

One Sentence Description

ProctorShield AI is a role-based web examination platform that lets teachers create and control quizzes, lets students take them on desktop or mobile browsers, records selected integrity events as short evidence clips, provides real-time session signals, and keeps final integrity decisions under human review.

Problem Being Addressed

Online quizzes are easy to distribute but difficult to supervise consistently. Teachers need one workflow for quiz creation, controlled start, live awareness, incident evidence, grading, retakes, and reports. Students need accessible participation without installing a native application. Administrators need user, subscription, and operational oversight.

Implemented Value

- Teacher, student, and administrator portals with server-enforced role boundaries.
- Manual quiz creation, AI-assisted creation for entitled teachers, short join codes, teacher-controlled start, autosave, grading, results, and retakes.
- Adaptive camera monitoring, browser-integrity events, object and face-related signals, short downloadable evidence clips, and AI-assisted report summaries.
- Private real-time channels for session starts, joins, violations, submissions, approvals, notifications, and teacher warnings.
- Free and Pro plan enforcement on the server, with PayMongo test-mode billing and persistent subscription state.
- PostgreSQL, Redis, private S3-compatible evidence storage, Google OAuth, email-based security flows, and operational preflight checks.

Audit Conclusion

Current engineering status. The project compiles, validates its Prisma schema, passes 33 unit and security tests, passes 14 public browser contract tests, and has previously passed the production build, dependency audit, deployment preflight, and local/public health checks. It is suitable for a controlled capstone demonstration.

Not yet defensible as unrestricted production deployment. Eight authenticated end-to-end scenarios remain skipped by default; a signed physical-device matrix, calibrated model evaluation, privacy governance, provider-failure tests, load evidence, and centralized production observability remain incomplete. The latest mobile and warning improvements are also still local until committed and pushed.

2 Problem Scope and Users

User	Primary responsibilities	Protected boundary
Student	Register or sign in, join with a code, wait for the teacher, accept rules, answer questions, receive warnings, submit, view results, and request a retake.	Cannot create or start quizzes, read other students' attempts, or access teacher evidence.
Teacher	Create and manage owned quizzes, start sessions, monitor participants, warn students, review evidence, manage retakes, view reports, and manage a plan.	Can act only on owned resources; paid features require an active entitlement.
Administrator	Review users, analytics, operational state, and permitted account actions.	Administrative routes require a valid Admin session and explicit checks.

System Boundaries

The application is browser based. It can observe only browser and camera signals that the operating system and browser expose. It cannot reliably prevent a second device, guarantee screenshot detection on every phone, inspect another application, or prove intent. Mobile sessions are therefore labeled Reduced Assurance when strict desktop capabilities are unavailable.

Success Criteria

- Quiz lifecycle state is controlled by the server, including join, start, active attempt, submission, grading, and retake.
- Unauthorized users cannot cross roles or access unowned resources by changing a URL or request body.
- Incidents preserve contextual evidence when the device supports capture.
- Teacher warnings and quiz-state changes arrive immediately when connected and recover through polling after missed real-time events.
- Free and Pro plan limits remain correct after logout, login, webhook retries, and rejected payment flows.

3 System Architecture

Layer	Technology	Responsibility
Web application	Next.js 16.3.4, React 19.2.4, TypeScript	Role portals, server routes, rendering, validation, quiz and monitoring user experience.
Application data	Prisma 7.10 and PostgreSQL on Neon	Users, quizzes, questions, attempts, answers, violations, evidence metadata, notifications, payments, and migrations.
Transient shared state	Redis through ioredis	Distributed rate limiting, live snapshots, warning recovery, and short-lived coordination.
Evidence objects	Private S3-compatible storage and local MinIO	Incident video objects kept outside the relational database; authorized routes mediate access.
Real time	Pusher private channels	Quiz starts, joins, warnings, violations, submissions, retake approvals, and notifications.
AI assistance	Gemini model fallback chain	Quiz drafting and report assistance; server rules and human review remain authoritative.
Identity and email	Credentials, Google OAuth, SMTP	Login, registration, MFA-related codes, and password recovery.
Billing	PayMongo test mode	Checkout and signed webhook processing for Pro entitlements.
Local services	Docker Compose	Redis and MinIO for reproducible development and testing.

Request Path

1. A browser calls a Next.js page or API route.
2. The server authenticates the role-specific session and validates the request.
3. Ownership, quiz state, subscription, and rate-limit rules are enforced on the server.
4. Prisma performs transactional PostgreSQL operations where consistency matters.
5. Redis, Pusher, storage, Gemini, email, or PayMongo is used only for the part of the workflow it owns.
6. The API returns a controlled response; the client updates immediately or reconciles through fallback polling.

Why This Architecture

Relational records need strong consistency, while video evidence is better stored as private objects. Real-time signals require a push channel but cannot be treated as durable truth, so database state and short polling remain the recovery path. External AI and payment providers are helpers, not the authority for quiz access, grading, or subscription activation.

4 Core Workflows

Teacher Quiz Lifecycle

1. The teacher creates a manual quiz or, when entitled, requests an AI-assisted draft.
2. The server validates the payload, ownership, question structure, plan limits, and access-code uniqueness.
3. The quiz remains Draft until the teacher starts it.
4. Starting transitions the quiz to In Progress and publishes a start event.
5. The teacher monitors joined students, incidents, answers, submissions, and warnings.
6. Results and evidence remain associated with the owning teacher and attempt.

Student Quiz Lifecycle

1. The authenticated student normalizes and submits the join code.
2. The server checks quiz existence, eligibility, lifecycle state, and attempt history.
3. If the teacher has not started the quiz, the student remains on a waiting screen.
4. After start, the student accepts rules, grants available permissions, and enters the quiz.
5. Answers are checked and recorded on the server; wrong feedback may be shown before advancing according to quiz behavior.
6. The final question triggers submission, or the student uses the visible submit action where provided.

Password Recovery

The recovery route accepts a role-aware request, sends a short-lived code through configured email, validates purpose and expiry, then updates the password using the same secure hashing policy. Responses should avoid exposing whether an arbitrary email address exists.

Evidence Lifecycle

A qualifying browser or camera incident creates a violation record and attempts to record a short clip, generally three to five seconds when the platform supports MediaRecorder. The object is uploaded to private storage, its metadata is linked to the attempt, and an authorized teacher can stream or download it through a guarded evidence route.

5 Proctoring and Evidence Design

Signal Categories

Signal	Meaning	Limitation
Face-related	Face missing, additional face, or position-related signals from available camera frames.	Lighting, occlusion, camera angle, skin-tone distribution, and device performance require formal validation.
Object-related	Detected objects such as a mobile phone in frames when the model and camera view permit it.	An object outside the frame or below the confidence threshold cannot be detected.
Browser integrity	Visibility changes, leaving fullscreen, focus changes, restricted actions, and related events.	Mobile operating systems expose fewer reliable events; screenshot detection is not universal.
Teacher observation	Live status, periodic snapshots, violations, answers, and manual warnings.	The current teacher monitor is adaptive snapshot monitoring, not guaranteed continuous video.
AI-assisted verdict	Gemini or fallback rules summarize observed events.	Current numeric output is not a scientifically calibrated probability and must not be treated as proof.

Evidence Controls

- Evidence objects are private rather than public URLs.
- Upload routes check authentication, ownership or attempt relationship, media type, size, and available signature or duration metadata.
- Downloaded or streamed evidence passes through an authorized server route.
- Retention is represented in implementation, but policy approval, disclosure, deletion requests, and appeal procedures still need formal governance.

Defense framing. A flag is an indicator requiring review. The teacher should consider the clip, event timeline, question context, environmental explanation, and the student's opportunity to respond.

6 Real Time and Mobile Design

Real-Time Event Model

Private Pusher channels carry low-latency session events. Durable state remains in PostgreSQL, while Redis stores selected short-lived recovery data. The student warning flow now combines immediate push delivery with approximately five-second polling so a missed event can still appear after reconnection.

Adaptive Device Profiles

Profile	Camera target	Detection interval	Snapshot interval
Desktop strict	640 × 480 around 24 fps	About 500 ms	About 2 seconds
Typical mobile	320 × 240 around 12 fps	About 1.8 seconds	About 5 seconds
Low-power mobile	320 × 240 around 10 fps	About 2.8 seconds	About 8 seconds

These profiles reduce heat, memory pressure, bandwidth use, and browser hangs. They also reduce temporal detail. Every mobile session should be treated as Reduced Assurance instead of being described as equivalent to a controlled desktop environment.

Current Live-Monitoring Boundary

The student code contains WebRTC signaling and sender-side elements, but the teacher monitor does not yet contain a complete RTCPeerConnection receiver path and no production TURN configuration is demonstrated. Therefore, defend the implemented feature as real-time events plus adaptive camera snapshots. Do not describe it as universally reliable continuous live video.

Supported-Device Answer

The system is designed for current Chromium-based Android browsers and modern iOS Safari, subject to camera permission, HTTPS, memory, network, and MediaRecorder support. It cannot honestly promise all Android and iOS devices. The correct acceptance criterion is a documented device-and-browser matrix with pass, reduced-assurance, or unsupported status.

7 Data Model

The audited Prisma schema contains 19 models. The important relationships are summarized below.

Domain	Representative records	Integrity purpose
Identity	User, role and session-related fields	Role isolation, account state, subscription state, and session invalidation.
Assessment	Quiz, Question, choices and ownership	Teacher ownership, unique join code, lifecycle state, and correct-answer definition.
Participation	StudentQuiz or attempt, answer records and status	Eligibility, attempt history, autosave, score, completion, and retake behavior.
Integrity	Violation, evidence metadata and report-related data	Incident timeline, private object references, review, and reporting.
Communication	Notification and live coordination records	Clickable destinations, read state, warnings, and workflow feedback.
Billing	Payment and subscription-related records	Idempotent provider events, amount and mode checks, and persistent entitlement.

Consistency Techniques

- Database uniqueness constraints protect identifiers and race-sensitive records.
- Transactions group dependent writes such as submission and grading operations.
- Advisory locking or equivalent serialization protects attempt-sensitive work.
- Server-side lifecycle predicates reject invalid state transitions even if the client UI is bypassed.
- Migrations, not ad hoc production edits, define durable schema evolution.

8 Security Model

Threat	Implemented control	Residual concern
Password compromise	bcrypt with cost 12, recovery codes with expiry and purpose binding.	Email account security and credential reuse remain user risks.
Session forgery	HS256 JWT verification with explicit algorithm allowlist and minimum 32-character secret requirement.	Secret rotation and production secret-manager evidence must be maintained.
Session theft	Role-specific HttpOnly cookies, SameSite Lax, Secure in production, session-version revocation.	Compromised endpoints and browser extensions remain outside full control.
Broken access control	Server-side role, ownership, resource, state, and subscription checks.	Authenticated end-to-end regression coverage is not yet complete.
Brute force and abuse	Redis-backed rate limits with a process-local development fallback.	The local fallback is not safe as a distributed production control; production readiness should require Redis.
Web injection and framing	Validation plus CSP, HSTS, frame denial, nosniff, referrer and permissions headers.	Headers and third-party origins require review whenever integrations change.
Unauthorized evidence access	Private storage, guarded retrieval, ownership checks, type, size, and metadata validation.	Institutional privacy approval and formal retention/deletion workflow remain incomplete.
Webhook forgery or replay	Raw-body HMAC, timestamp, mode, amount, idempotency, and refund validation.	Provider outage and reconciliation drills should be documented.

Security Principles Demonstrated

- The server is authoritative; disabled buttons are usability, not authorization.
- External redirects do not grant paid access. Only a validated payment event can change entitlement.
- Secrets belong in environment configuration. The tracked example contains placeholders rather than production values.
- Evidence and personal data are minimized to the system purpose and should be governed by explicit policy.

9 Subscription and Billing

Capability	Free teacher	Pro teacher
Manual quiz creation	Limited to five quizzes under the current rule.	Unlimited under the application entitlement.
AI Create	Blocked by server entitlement checks.	Available subject to provider configuration and limits.
Live Monitoring	Blocked by server entitlement checks.	Available with the documented browser and device limitations.
Persistence after login	Remains Free until a valid upgrade.	Stored in the database and restored across logout and login.

PayMongo remains in test mode for the capstone. A return URL is not proof of payment. The signed webhook is validated for mode, event authenticity, expected amount, replay safety, and relevant refund state before the application changes the subscription.

10 Verification Evidence

Verification	Latest observed result	Interpretation
ESLint	Passed with quiet mode	No current lint errors were reported.
TypeScript	Passed with incremental mode disabled	The audited source type-checks.
Prisma validation	Passed	The configured schema is structurally valid.
Unit and security tests	33 of 33 passed	Core predicates, validation, submission, and security contracts pass.
Browser contract tests	14 passed, 8 skipped	Public desktop and Pixel 7 emulation paths pass; authenticated flows need the explicit disposable-database run.
Production build	Previously passed after current real-time changes	Next.js compiled the application and route set successfully.
Dependency audit	Previously 0 known production vulnerabilities	This is point-in-time package evidence, not a permanent guarantee.
Operational preflight	Passed	Database, Redis, evidence storage, and critical uniqueness checks were reachable and valid.
Database restore	Snapshot restored to a separate Neon branch and row counts inspected	A recoverable backup path has been exercised once.

What the Test Numbers Do Not Prove

Passing automated tests does not establish model accuracy, performance under class-sized load, every physical phone/browser combination, long-duration stability, privacy compliance, or behavior during third-party outages. Those require separate evidence.

11 Prioritized Audit Findings

Priority	Finding	Why it matters	Required action
1 High	Unsupported accuracy marketing claims remain in UI copy, including 98.4 percent detection accuracy, 99 percent accurate face detection, and 100 percent verified integrity.	The repository contains no labeled study supporting those percentages.	Remove or qualify the claims. Run a representative labeled evaluation and report precision, recall, false-positive rate, false-negative rate, sample size, device, lighting, and threshold.
2 High	Latest mobile performance and real-time warning improvements are local and uncommitted.	A demonstration or deployment from GitHub may not contain the tested behavior.	Review the diff, rerun verification, commit, push, and verify the remote commit before defense.
3 High	Eight authenticated browser scenarios and a signed physical-device matrix remain incomplete.	The highest-risk workflows depend on real sessions, roles, database state, camera permissions, and two devices.	Run authenticated Playwright tests against a disposable database, then execute a controlled Android/iOS and teacher-laptop matrix with evidence.
4 High	The AI verdict exposes a value described like a cheating probability without calibration evidence.	A rule-derived or model-generated score can be mistaken for scientific certainty or proof.	Rename it to risk score or review priority, disclose inputs, preserve human review, and validate before using probabilistic language.
5 Major	Live camera wording exceeds the verified implementation.	Teacher monitoring currently relies on adaptive snapshots; a complete receiver and TURN path is not demonstrated.	Use accurate UI and defense wording, or finish WebRTC receiver, TURN, reconnect, and network-failure testing.
6 Major	Privacy, consent, retention, deletion, incident review, and appeal governance are incomplete.	Evidence includes sensitive camera and behavioral data.	Publish clear notices, obtain institutional review, define retention and deletion jobs, document access logging and appeals, and obtain legal/privacy review appropriate to deployment.
7 Major	External-provider failure, load, scheduler, maintenance, and recovery evidence is incomplete.	Gemini, Pusher, SMTP, storage, Redis, PayMongo, and Neon can fail independently.	Run timeout, retry, duplicate, outage, degraded-mode, concurrency, and recovery drills with recorded outcomes.
8 Moderate	Several UI modules are very large and status values are partly free-form.	The quiz page is about 2,365 lines, teacher quiz content about 1,246, and monitor content about 756, increasing regression risk.	Extract state machines, media capture, detection, real-time transport, and view components. Centralize lifecycle enums and transition rules.

Priority	Finding	Why it matters	Required action
9 Moderate	Observability is mostly application logs.	Production failures may be noticed late and reconstructed poorly.	Add centralized exception tracking, uptime checks, structured logs, alert ownership, and privacy-safe operational metrics.
10 Minor	README mobile snapshot timing is older than the current adaptive profile.	Documentation can mislead testers and panelists.	Update the README to approximately five seconds for typical mobile and eight seconds for low-power mobile.

Lower-Level Technical Debt

- Maintenance deletes an evidence object before a related database transaction; a database failure afterward may leave stale metadata. Prefer a retryable deletion queue or mark-and-sweep sequence.
- A process-local rate-limit fallback is acceptable for isolated development but not for multi-instance production. Fail readiness when Redis is required and unavailable.
- A hard-coded fallback Pusher public key is not a secret, but it can connect the wrong environment silently. Production should fail clearly when the expected public configuration is absent.

12 Claims to Avoid During Defense

Avoid saying	Use this defensible wording
It prevents cheating.	It detects selected risk signals, captures supporting evidence, and helps a teacher review incidents.
It is 99 percent accurate.	The implementation works in controlled tests; formal accuracy measurement on a labeled representative dataset is future validation work.
The score proves the student cheated.	The score prioritizes review and is not a verdict. The teacher reviews the evidence and context.
It detects every screenshot or minimized app on mobile.	It uses the events exposed by the browser. Mobile operating systems impose limitations, so mobile is Reduced Assurance.
It supports all Android and iOS devices.	It targets current Android Chromium browsers and modern iOS Safari, subject to a documented compatibility matrix.
The teacher watches continuous live video.	The current monitor delivers real-time events and adaptive camera snapshots; continuous WebRTC viewing remains incomplete.
It is completely secure and production ready.	Layered controls and automated checks are implemented. Controlled deployment still requires device UAT, privacy governance, observability, load and failure testing.
It is legally compliant.	The design includes privacy controls, but legal and institutional compliance must be reviewed for the deployment context.

13 Demonstration Plan

Before the Panel Arrives

1. Confirm the exact Git commit and preserve a clean demo branch.
2. Start Docker Redis and MinIO, run the preflight, then run the production build or approved server command.
3. Verify database connectivity, private evidence storage, Pusher, email, Gemini, Google OAuth, and PayMongo test configuration without displaying secrets.
4. Prepare a teacher account, student account, Free-plan account, Pro-plan account, a two-question quiz, and one pre-recorded evidence example.
5. Keep a local fallback recording or screenshots in case the school network blocks an external provider.

Eight-Minute Live Demo

Time	Action	Point to explain
0:00–0:45	Show portal selection and role-specific login.	Authentication and role isolation are enforced on the server.
0:45–1:45	Create a short manual quiz and show its code.	Free limit, server validation, ownership, and short-code normalization.
1:45–2:30	Join from a phone before start.	The student waits because the database state, not the button, controls access.
2:30–3:15	Start from the teacher laptop.	The phone changes through a real-time event with recovery polling.
3:15–4:30	Answer questions on the phone.	Responsive mobile UI, server-recorded answers, feedback, advance, and final submission.
4:30–5:15	Send a teacher warning.	Immediate Pusher delivery plus Redis-backed missed-event recovery.
5:15–6:20	Trigger or show a safe prepared incident.	Short evidence clip, private storage, teacher authorization, and human review.
6:20–7:10	Show result, report, and retake behavior.	Attempt history and server-side lifecycle enforcement.
7:10–8:00	Show Free versus Pro gating and test billing.	Entitlement persists in the database; a redirect alone cannot activate Pro.

Failure Recovery Script

If an external service fails, state which dependency failed, show that core database state remains authoritative, use prepared evidence, and continue. Do not hide the failure. Explain the designed fallback and the specific test still required.

14 Defense Presentation Script

Opening

“Good day. ProctorShield AI addresses the supervision gap in browser-based online quizzes. It combines teacher-controlled assessment workflow, adaptive integrity signals, short incident evidence, real-time communication, and human review. Our design does not claim to eliminate cheating. It helps teachers identify, document, and review selected risks while preserving server-side control of the assessment lifecycle.”

Architecture

“The system uses Next.js and TypeScript for the web application, Prisma with PostgreSQL for durable relational state, Redis for shared transient controls and recovery, private S3-compatible storage for evidence, and Pusher private channels for low-latency events. Gemini assists with drafting and summaries, but it is not authoritative. PayMongo is intentionally in test mode.”

Security

“We treat the client as untrusted. The server checks session role, ownership, quiz state, subscription, and input constraints. Passwords are hashed with bcrypt. JWT verification restricts the accepted algorithm, cookies are HttpOnly and secure in production, rate limits use Redis, evidence is private, and billing activation requires a signed idempotent webhook.”

Testing

“The latest audit passed lint, TypeScript, Prisma validation, 33 unit and security tests, and 14 public browser tests. A production build, dependency audit, operational preflight, and backup restore were also exercised. We explicitly report that eight authenticated browser scenarios and the full physical-device matrix remain incomplete.”

Limitations and Next Work

“Mobile browsers cannot provide the same assurance as a desktop environment, continuous live video is not yet fully implemented, and model accuracy claims require a labeled study. Our next work is to complete authenticated and physical-device testing, calibrate the detection evaluation, formalize privacy and appeals, validate provider failures and load, and add centralized observability.”

Closing

“The contribution of ProctorShield AI is an integrated, server-controlled assessment and review workflow that is transparent about uncertainty. The project demonstrates practical full-stack engineering and establishes a measurable path from capstone prototype to controlled deployment.”

15 Likely Panel Questions

1 What makes this different from a normal quiz application?

It integrates teacher-controlled start, adaptive integrity signals, evidence capture, real-time warnings, incident review, retakes, reporting, and plan enforcement in one role-based workflow.

2 Does the system prove cheating?

No. It flags observable indicators and preserves context. A human reviewer must decide whether the event has an innocent explanation or requires action.

3 Why use AI?

AI assists with quiz drafting and incident summarization. It reduces repetitive work but never replaces server rules, grading authority, or the teacher's final review.

4 How did you measure detection accuracy?

A formal representative labeled evaluation is not yet present, so percentage claims should not be defended. The required study must publish dataset conditions, thresholds, precision, recall, false-positive and false-negative rates.

5 Why is mobile Reduced Assurance?

Mobile browsers restrict fullscreen, background behavior, focus visibility, screenshot signals, sustained camera workloads, and memory. The adaptive profile keeps the quiz usable but cannot provide desktop-equivalent control.

6 What happens if Pusher misses an event?

Durable state stays in PostgreSQL. Selected warning state is recoverable through Redis-backed polling, so the client can reconcile after reconnecting.

7 Why use both PostgreSQL and object storage?

PostgreSQL provides relationships and transactions for application records. Large video evidence belongs in private object storage, while the database keeps controlled metadata and ownership links.

8 How is role access protected?

Role-specific sessions are checked on server routes together with resource ownership and lifecycle rules. Hiding a button is never treated as authorization.

9 What prevents a student from entering before the teacher starts?

The join and quiz routes validate the authoritative quiz lifecycle state. The waiting interface mirrors that server rule but does not replace it.

10 How do you protect evidence?

Evidence is stored privately and retrieved through an authenticated ownership-aware route. Uploads are constrained by type, size, relationship, and available media validation.

11 What if a payment redirect is forged?

The return page cannot activate Pro. Subscription changes require a correctly signed PayMongo webhook whose mode, amount, timestamp, event identity, and refund state pass validation.

12 Why Redis?

Redis provides shared short-lived state for rate limits, snapshots, and warning recovery. A process-local fallback cannot coordinate multiple production instances.

13 Is Google OAuth secure on a temporary tunnel?

Only when the exact HTTPS callback is registered and secrets remain server-side. Temporary domains are for controlled testing, not a stable production identity endpoint.

14 Can it support a whole class?

The architecture can scale components independently, but a class-size claim needs measured concurrent-user, latency, bandwidth, storage, and provider-limit tests. That formal evidence remains pending.

15 What is your strongest security design choice?

Server-authoritative state and layered authorization. Even if a client is modified, the server still validates role, ownership, lifecycle, entitlement, and request structure.

16 Additional Technical Questions

16 Why JWT cookies rather than local storage?

HttpOnly cookies reduce direct JavaScript access to the token. Secure production transport, SameSite behavior, short validity, and session-version checks complement that choice.

17 How do retakes avoid overwriting history?

Attempts are represented as history rather than replacing the earlier result. Approval and eligibility are enforced as explicit state transitions.

18 How do you handle race conditions during submission?

Server predicates, database transactions, uniqueness constraints, and serialization mechanisms ensure that repeated or concurrent requests do not create conflicting final outcomes.

19 Why not store video in PostgreSQL?

Binary media would increase database size, backup cost, and query pressure. Private object storage is designed for large objects while PostgreSQL keeps searchable metadata.

20 What happens when Gemini is unavailable?

Core manual quizzes, access control, taking, grading, and evidence records should continue. AI-only assistance must return a controlled error or fallback rather than corrupting assessment state.

21 What is wrong with calling the verdict a probability?

A probability must be calibrated against representative labeled outcomes. A fixed rule or generative model response is better described as a risk score until calibration is demonstrated.

22 How do you test mobile?

Browser emulation catches layout and contract issues but is not enough. Use physical Android and iOS devices across browsers, network conditions, camera permissions, interruptions, low memory, and long sessions.

23 What is the backup strategy?

Neon history and snapshots protect database state, and a restore to a separate branch has been exercised. Object-storage backup, recurring restore evidence, recovery targets, and ownership still require formal documentation.

24 What personal data is processed?

Identity, quiz activity, results, integrity events, and potentially face-containing camera evidence. Collection, access, retention, deletion, and appeals must be explained before participation.

25 How do you reduce false accusations?

Use multiple signals, preserve clips and timelines, show uncertainty, prohibit automatic punishment, give students a response channel, and evaluate false-positive rates across conditions.

26 Why are authenticated tests skipped by default?

They mutate real account and assessment state, so they require an explicit flag and disposable database. The safe design is correct, but the suite must still be run and recorded before claiming full end-to-end coverage.

27 What would you refactor first?

Split the large quiz page into lifecycle state, media capture, detection, real-time transport, answer workflow, and presentation modules, each with focused tests.

28 What is needed for continuous live video?

A complete teacher-side WebRTC receiver, TURN service, authentication for signaling, reconnect logic, bandwidth controls, multi-student layout, privacy rules, and network-failure testing.

29 Is Docker required for end users?

No. Docker is a development and deployment tool for services such as Redis and MinIO. Students and teachers use the web application through a supported browser.

30 What is the most important next milestone?

Create a reproducible release candidate: commit current fixes, run all authenticated tests, complete the physical-device matrix, record results, and remove unsupported claims before the final defense.

17 Final Defense Checklist

Area	Required evidence	Status at audit
Source control	Named release commit on GitHub containing current fixes.	Pending for latest local changes
Static quality	Lint, TypeScript, Prisma validation, production build logs.	Passed
Automated behavior	33 unit/security tests plus all authenticated browser tests.	Unit passed; 8 authenticated scenarios pending
Physical devices	Signed Android, iOS, and teacher-laptop matrix with screenshots and incident clips.	Pending
Accuracy	Labeled test protocol and confusion-matrix metrics.	Pending
Privacy	Notice, consent, retention, deletion, access, review, and appeal procedure.	Partly implemented; governance pending
Operations	Preflight, health checks, structured monitoring, provider failure and load evidence.	Preflight passed; remaining evidence pending
Recovery	Database and evidence restore drills with recovery time and result.	Database branch restore exercised once
Demo	Prepared accounts, short quiz, charged devices, stable URL, backup recording.	Prepare immediately before defense
Claims	Slides and UI avoid unsupported accuracy, universality, proof, and legal claims.	UI cleanup required

Go or No-Go Rule

No-go for an unrestricted claim if authenticated tests, physical-device results, privacy approval, or accuracy evidence are missing. **Go for a controlled capstone demonstration** when the exact release commit passes preflight and automated checks, the prepared two-device workflow succeeds, limitations are disclosed, and backup demo evidence is ready.

18 Command Reference

Purpose	Command
Start local dependencies	<code>docker compose up -d redis minio minio-init</code>
Inspect containers	<code>docker compose ps</code>
Operational readiness	<code>npm run preflight</code>
Lint	<code>npm run lint -- --quiet</code>
Type check	<code>npx tsc --noEmit --incremental false</code>
Validate data schema	<code>npx prisma validate</code>
Unit and security tests	<code>npm test</code>
Public browser contracts	<code>npm run test:e2e</code>
Authenticated browser tests	<code>\$env:RUN_AUTHENTICATED_E2E='true'; npm run test:e2e</code> against an explicitly disposable database
Production build	<code>npm run build</code>
Production dependency audit	<code>npm audit --omit=dev --audit-level=high</code>
Inspect source status	<code>git status</code>

Never display actual secrets in terminal output, screenshots, slides, documents, or source control. Authenticated tests must never target the production database.

19 Glossary

Term	Meaning in ProctorShield AI
Server authoritative	The server and database decide valid state even when the browser requests something different.
RBAC	Role-based access control for Student, Teacher, and Admin operations.
JWT	Signed session token stored in an HttpOnly role-specific cookie.
OTP	Short-lived one-time code for multi-factor login or password recovery.
ORM	Object-relational mapping; Prisma maps TypeScript operations to PostgreSQL.
Transaction	A group of database changes that succeed or fail together.
Advisory lock	A PostgreSQL lock used to serialize race-sensitive work for one attempt or resource.
Idempotency	Processing the same external event again does not duplicate its business effect.
Redis	Shared in-memory service for short-lived state and distributed rate limits.
S3-compatible storage	Private object storage for media evidence rather than database binary fields.
Pusher	Hosted private-channel service for connected-browser events.
MediaRecorder	Browser API used to create short incident video clips when supported.
Reduced Assurance	The session can run, but device or browser restrictions prevent the strict desktop control set.
False positive	An innocent event is flagged as suspicious.
False negative	A suspicious event occurs but the detector does not flag it.
Precision	The proportion of flagged labeled examples that are actually positive.
Recall	The proportion of actual positive labeled examples that are detected.
Preflight	Operational check for database, Redis, evidence storage, and critical constraints before deployment.
TURN	A relay service required for reliable WebRTC media when direct network paths fail.
UAT	User acceptance testing by representative students, teachers, and administrators.

Final study reminder. The strongest defense is accurate. ProctorShield AI has meaningfully implemented controls and passing automated evidence. Its limitations do not invalidate the project; they define the next responsible engineering and research work. Explain what the system enforces, show the evidence, admit what a browser cannot guarantee, and present a concrete validation plan.

